



Fall Semester 2025-26 – Case Study- 1

Course Code: BECE102L

Due Date : 15th October 2025

Course Name: Digital Systems Design

Max. Marks : 10

Class ID: VL2025260103643

Slot : F1+TF1

Registration Number	Case Study: Traffic Light Controller
21BCE2093 22BCB0189 24BAI0035 24BAI0038 24BAI0058	A city intersection uses a digital control system to manage traffic lights. The traffic lights can be in one of three states: Green, Yellow, or Red. There are two directions, North–South and East–West, and only one direction can have a green light at a time. Questions: - Draw the state diagram for the traffic light controller. - Create a truth table for the transitions between states. - Design the sequential circuit using flip-flops. - How would you implement safety logic to avoid conflicting green signals? - If a pedestrian button is pressed, the system should complete its current cycle before allowing pedestrian crossing. How would you modify the design?

Registration Number	Case Study: 4-Bit Binary Adder
24BAI0065 24BAI0070 24BAI0090 24BAI0121 24BAI0132	You are tasked with designing a 4-bit binary adder to add two 4-bit numbers with carry-in and carry-out. Questions: - Explain how a full adder works and how it differs from a half adder. - Design the circuit diagram for a 4-bit adder using full adders. - Write the Boolean expressions for the sum and carry outputs. - How would you detect an overflow condition in the addition? - How could you expand this design to an 8-bit adder?

Registration Number	Case Study: Digital Lock System
24BAI0163 24BAI0223 24BAI0235 24BCB0071 24BCB0110	A security system uses a digital keypad where a correct sequence of 4 digits must be entered to unlock the door. The system uses flip-flops and logic gates to store and compare input. Questions: - Design the state machine to represent entering the passcode. - How would you implement debounce logic for the keypad inputs? - Explain how you would store the correct passcode and compare the entered sequence. - How can you reset the system after a failed attempt? - Propose how you would add a feature to lock out the system after 3 failed attempts.

Registration Number	Case Study: Binary to Gray Code Converter
24BCB0150 24BCE0063 24BCE0287 24BCE0403 24BCE0427	You need to design a circuit that converts a 4-bit binary number into its Gray code equivalent. Questions: - What is the rule for converting binary to Gray code? - Derive the logic expressions for each output bit. - Draw the logic circuit diagram. - Why is Gray code used in digital communication or error correction systems? - How would you implement the reverse conversion (Gray to Binary)?

Registration Number	Case Study: Digital Stopwatch
24BCE0443 24BCE0525 24BCE0535 24BCE0572 24BCE0895	Design a digital stopwatch that counts up in seconds from 00 to 59 and resets to 00 after reaching 59. Questions: - Identify the counters needed for seconds and minutes. - Write the truth table for the counter transitions. - How would you implement the clock pulse generator? - How can you design the display to show the count in 2-digit format? - Suggest how you would add features like “Pause” and “Reset.”

Registration Number	Case Study: 4-bit Arithmetic Unit
24BCE0928 24BCE0942 24BCE0972 24BCE2002 24BCE2077	Design a 4-bit arithmetic unit that performs addition, subtraction, increment, and decrement based on a 2-bit control signal. Questions: - Create the truth table and operation selection logic. - Write the Verilog module using a case statement. - How will you ensure overflow detection? - Write a testbench to validate all operations. - Explain how timing and propagation delay affect the circuit’s performance.

Registration Number	Case Study: Password Lock System
24BCE2082 24BCE2161 24BCE2206 24BCE2211 24BCE2285	Design a password lock system that unlocks when a correct sequence is entered. Questions: - Model the states for password entry using a state machine. - Use Verilog case statements to handle state transitions. - How would you reset the state machine after incorrect input? - Implement a timer that locks the system after 10 seconds of inactivity. - Describe how you would test the system for boundary cases.

Registration Number	Case Study: Simple ALU with Multiple Operations
24BCE2346 24BCE2368 24BCE2504 24BCE2530 24BCE2532	Create an Arithmetic Logic Unit (ALU) that takes two 8-bit inputs and performs eight different operations based on a 3-bit opcode. Questions: - List the operations and map them to opcode values. - Implement the ALU in Verilog using a case statement. - How would you extend the design to include shift operations? - Write test cases to confirm correct output for each opcode. - Discuss how you would pipeline the ALU for higher speed.

Registration Number	Case Study: Digital parking garage gate controller
24BCE2553 24BCE2609 24BCE2700 24BCE2707 24BCE2827	Scenario You are tasked with designing a logic circuit for a small parking garage with four parking spaces. The gate should only open if a car is present at the entrance and there is at least one empty parking space. An "OPEN" signal should be generated to control the gate. Inputs Entrance Sensor (E): A logic '1' indicates a car is at the entrance; a logic '0' indicates no car. Space 1 Occupied (S_1): A logic '1' indicates the space is occupied; a logic '0' indicates it is empty. Space 2 Occupied (S_2): A logic '1' indicates the space is occupied; a logic '0' indicates it is empty. Space 3 Occupied (S_3): A logic '1' indicates the space is occupied; a logic '0' indicates it is empty. Space 4 Occupied (S_4): A logic '1' indicates the space is occupied; a logic '0' indicates it is empty. Output Gate Open (G): A logic '1' opens the gate; a logic '0' keeps it closed. Questions Truth Table Creation: Create a truth table that defines the output (G) for all possible combinations of the input variables (E, S_1, S_2, S_3, S_4). Boolean Expression: Write the canonical Sum-of-Products (SOP) boolean expression for the output G. Karnaugh Map Minimization: Use a Karnaugh map (K-map) to simplify the boolean expression from the previous step. Logic Diagram: Draw the logic circuit diagram using basic logic gates (AND, OR, NOT) based on the simplified expression.

Registration Number	Case Study: BCD to excess-3 code converter
24BCE2838 24BCE2864 24BCE2941 24BCE2948 24BCE2970	A digital system requires a circuit to convert 4-bit Binary Coded Decimal (BCD) numbers into their corresponding Excess-3 code. BCD is a 4-bit representation of decimal digits 0 through 9. Excess-3 code is another 4-bit code where each decimal digit is represented by its BCD equivalent plus three. Inputs 4-bit BCD Input (B_3, B_2, B_1, B_0) Outputs 4-bit Excess-3 Output (E_3, E_2, E_1, E_0) Questions Truth Table Creation: Create a truth table that shows the BCD inputs and the required Excess-3 outputs. Include "Don't Care" conditions for the BCD inputs that are not valid decimal digits (1010 to 1111). Karnaugh Map Minimization: Use K-maps to simplify the boolean expression for each of the four Excess-3 outputs (E_3, E_2, E_1, E_0). Circuit Implementation: Draw the final combinational logic circuit using logic gates. Alternative Implementation: Propose an alternative implementation for this circuit using a 4-bit binary adder and justify your choice.

Registration Number	Case Study: ALU with status flags
24BCI0160 24BCI0163 24BCI0297 24BCT0101 24BCT0161	Scenario: Design a 16-bit Arithmetic Logic Unit (ALU) capable of performing addition, subtraction, AND, and OR operations. The ALU must also produce status flags (Zero, Negative, Carry-Out, and Overflow) for use by the control unit. Questions: ALU design: Draw the block diagram for the 16-bit ALU. Show the inputs (operands, function select) and outputs (result, status flags). Status flag logic: Provide the Boolean expressions or logic for generating each of the four status flags (Zero, Negative, Carry-Out, Overflow) based on the ALU's result and inputs. Data-path integration: Integrate this ALU into a multi-cycle Data-path that supports a conditional branch instruction (BZ , branch if Zero). Control unit dependency: Explain how the control unit would use the Zero flag generated by the ALU to correctly execute the BZ instruction.

Registration Number	Case Study: Vending Machine Finite State Machine (FSM) Design
24BCT0182 24BCT0183 24BCT0286 24BCT0334 24BDS0028	Task: Design an FSM to control a vending machine that accepts coins and dispenses products. Considerations: - States for accepting different coins (e.g., 1 Rupee, 2 Rupees). - States for product selection. - States for dispensing the product and calculating change. - Outputs for product dispensing and change delivery. - Handling insufficient funds.

Registration Number	Case Study: Traffic light controller
24BDS0064 24BDS0075 24BDS0083 24BDS0107 24BDS0153	Design a simplified traffic light controller for a four-way intersection. The lights should cycle through the following sequence: State 0: North-South (N/S) green, East-West (E/W) red. State 1: N/S yellow, E/W red. State 2: N/S red, E/W green. State 3: N/S red, E/W yellow. The controller has a clock signal and an input (T) for a timer that goes high when the designated time for a light state has elapsed. Questions State diagram: Draw the state diagram for this controller. Define the state transitions based on the clock and timer signal (T). State assignment: Assign binary values to each of the four states. State table and excitation equations: Using D flip-flops, create the state table and derive the minimized Boolean expressions for the D-inputs and the four output signals (N/S Green, N/S Yellow, E/W Green, E/W Yellow). Asynchronous design consideration: Discuss the advantages and disadvantages of using a synchronous (clock-driven) versus an asynchronous design for a real-world traffic light controller.

Registration Number	Case Study: Parking garage exit counter
24BDS0227 24BDS0236 24BDS0368 24BKT0016 24BKT0077	Scenario: A small parking garage needs a system to count cars as they exit. The garage exit has two sensors: one at the front (A) and one at the back (B). A car's direction can be determined by the order in which the sensors are triggered: • Car exits: A then B. • Car enters (wrong way): B then A. • Noise/other blockage: A only or B only. Questions: - Boolean logic: Create the Boolean expressions for a logic circuit that produces a "decrement" signal only when a car correctly exits the garage. - PLD selection: Which type of simple PLD (PROM, PLA, or PAL) is most suitable for implementing this logic? Justify your choice based on the device's architecture. - PLA/PAL design: Draw the internal diagram of the selected PLD, showing how the programmable and fixed arrays are configured to implement the logic from question 1. - Hardware implementation: How would you incorporate a simple counter into this PLD design to track the total number of cars exiting? Describe the required sequential logic elements.